



RAPPORT DE PROJET – LICENCE 3 INFORMATIQUE

Université Le Havre Normandie

Squadro

Conception d'un jeu web persistant, stabilisation de
l'interface et enrichissement tactique

Auteur

Idris Achabou

Projet realise en L3 Informatique



Resume

Ce rapport presente la conception et l'evolution d'une implementation web du jeu **Squadro**. Le travail s'inscrit dans un contexte de Licence 3 Informatique a l'Universite Le Havre Normandie et porte a la fois sur la realisation fonctionnelle du jeu, la mise en place d'un environnement reproductible par Docker, la persistance des parties dans PostgreSQL et l'amelioration de l'experience utilisateur. L'application repose sur une architecture PHP volontairement lisible : un noyau de modele pour les pieces, le plateau et les actions, un controleur applicatif pour l'orchestration des coups, une couche d'accès aux donnees par PDO et une interface HTML/CSS/JavaScript separee du moteur de regles.

Le projet initial a ete consolide autour de plusieurs objectifs techniques : fiabiliser l'initialisation des sessions, supprimer les sorties parasites avant les redirections HTTP, serialiser correctement l'etat du plateau, isoler les styles de l'interface, stabiliser la position du plateau apres chaque action et enrichir le gameplay par des statistiques, un historique, une pile d'annulation et un module d'analyse tactique appele *Oracle*. Le theme graphique grec et spartiate ne constitue pas une simple decoration : il sert de fil conducteur a la lisibilite des ecrans, du lobby a l'ecran de victoire, tout en laissant le code metier independant de cette presentation.

Le rapport insiste enfin sur les arbitrages de maintenance : absence volontaire de framework lourd, centralisation du bootstrap, script SQL idempotent, tests de fumee, verification syntaxique, requetes AJAX pour eviter les rechargements brutaux et garde-fous de securite elementaires. L'ensemble forme un prototype jouable et documente, tout en conservant des pistes d'evolution realistes : authentication, matchmaking, WebSocket, migrations versionnees et tests unitaires plus complets.

Table des matières

1	Introduction	1
2	Manuel d'utilisation et d'exécution	2
2.1	Lancement par Docker Compose	2
2.2	Accès aux services	2
2.3	Identifiants de développement	2
2.4	Parcours fonctionnel	2
2.5	Raccourcis clavier	3
3	Regles du jeu et modelisation du domaine	4
3.1	Principe general	4
3.2	Etat d'une piece	4
3.3	Vitesses et trajectoires	4
3.4	Renvoi d'une piece adverse	4
4	Architecture logicielle	5
4.1	Lecture generale	5
4.2	Arborescence commentee	5
4.3	Role du bootstrap	6
4.4	Controleur principal	6
5	Persistance PostgreSQL	7
5.1	Motivation	7
5.2	Schema relationnel	7
5.3	Etats d'une partie	7
5.4	Script SQL idempotent	7
5.5	Synchronisation apres un coup	7
6	Moteur de jeu et invariants	9
6.1	Application d'un coup	9
6.2	Pile d'annulation	9
6.3	Controle de victoire	9
6.4	Gestion des erreurs	9
7	Oracle tactique et enrichissement du gameplay	10
7.1	Raison d'etre	10
7.2	Evaluation d'un coup	10
7.3	Mode contre l'Oracle	10
7.4	Panneaux de jeu enrichis	10
8	Interface, theme et stabilisation du plateau	12
8.1	Theme grec et spartiate	12
8.2	Correction du defaut de scroll	12
8.3	Mode confirmation fixe	12
8.4	Accessibilite minimale	12

9	Qualite, verification et maintenabilite	13
9.1	Strict types et centralisation	13
9.2	Echappement des sorties	13
9.3	Controle syntaxique	13
9.4	Smoke tests	13
9.5	Limites de la couverture	13
10	Problemes corriges pendant l'évolution	14
10.1	Sortie avant les headers	14
10.2	Serialization du plateau	14
10.3	Confirmation trop basse dans la page	14
10.4	Plateau instable apres un coup	14
11	Bilan critique et perspectives	15
11.1	Bilan	15
11.2	Perspectives techniques	15
11.3	Apport pedagogique	15
A	Annexe A - Matrice des fichiers principaux	16
B	Annexe B - Commandes de verification	17
C	Annexe C - Source institutionnelle du logo	18

Table des figures

1	Représentation de l'accueil spartiate : un point d'entrée simple, centre sur l'identité du joueur et l'entrée dans l'arène.	3
2	Flux architectural principal : l'interface déclenche les actions, le contrôleur valide, le modèle applique, la persistance synchronise.	5
3	Modèle relationnel minimal : une partie référence un ou deux joueurs et conserve l'état logique du plateau.	7
4	Pipeline d'exécution d'un coup.	9
5	Représentation de l'écran de partie enrichi : plateau central, Oracle, historique, commandes rapides et stabilisation visuelle.	11

Liste des tableaux

1	Services exposés par l'environnement Docker.	2
2	Matrice des interactions clavier principales.	3
3	Structure conceptuelle d'une pièce Squadro.	4
4	Cycle de vie persistant d'une partie Squadro.	7
5	Éléments pris en compte par l'analyse tactique.	10
6	Rôle des principaux fichiers du projet.	16

Listings

CHAPITRE 1

Introduction

Le projet **Squadro** consiste à mettre en œuvre un jeu de plateau abstrait sous la forme d’une application web. Le sujet est intéressant pédagogiquement car il combine trois dimensions souvent traitées séparément dans les exercices courts : la modélisation d’un domaine à règles précises, la gestion d’un état applicatif sur plusieurs requêtes HTTP, et la construction d’une interface suffisamment robuste pour ne pas perturber le jeu lui-même.

Dans Squadro, le joueur ne manipule pas un objet graphique libre : il choisit l’une de ses pièces et le moteur détermine une trajectoire selon la vitesse associée à la ligne ou à la colonne. Une pièce effectue un aller, pivote, puis revient vers sa zone d’origine. La victoire dépend de la capacité à terminer quatre trajets complets. Cette règle simple en apparence impose pourtant plusieurs invariants : une pièce doit avoir une couleur cohérente, une direction, une position valide, une vitesse applicable, un statut de sortie et un effet possible sur les pièces adverses rencontrées.

L’objectif du travail n’a donc pas été uniquement de produire un écran de jeu. Il s’agissait de rendre l’application exploitable dans un environnement standardisé, de clarifier les responsabilités du code et de corriger les défauts d’ergonomie qui rendent un jeu web pénible à utiliser. L’une des corrections les plus visibles concerne la stabilité du plateau : après un coup, le plateau ne doit pas descendre dans la page, même lorsqu’une barre de confirmation est activée. Cette contrainte d’interface a conduit à introduire une stratégie AJAX et une restauration contrôlée du viewport.

Objectifs directeurs du projet

- fournir une version directement exécutable par Docker sans installation locale de PostgreSQL ;
- conserver un moteur PHP lisible, découpé autour des notions de pièce, plateau, action et partie ;
- persister les parties pour permettre l’attente d’un second joueur et la reprise d’un état ;
- améliorer la jouabilité par un déplacement direct, une barre de confirmation fixe et des raccourcis clavier ;
- enrichir le jeu par un Oracle tactique, un historique, des statistiques et un export JSON ;
- appliquer une identité visuelle grecque/spartiate sans coupler le thème graphique aux règles du jeu.

Le document adopte une lecture progressive : manuel d’exécution, règles du jeu, architecture logicielle, persistance, moteur d’action, interface, qualité et perspectives.

CHAPITRE 2

Manuel d'utilisation et d'exécution

2.1 Lancement par Docker Compose

L'application est livrée avec un fichier docker-compose.yml. Le composeur instancie trois services : le serveur PHP/Apache, la base PostgreSQL et Adminer. Cette organisation évite les écarts d'environnement entre machines de développement.

Demarrage complet avec reconstruction des images

```
docker compose down -v
docker compose up --build
```

L'option -v supprime les volumes associés. Elle est utile lorsque l'on souhaite repartir d'une base propre. Pour conserver les parties déjà stockées, il suffit de lancer :

Demarrage sans suppression des données

```
docker compose up --build
```

2.2 Accés aux services

Service	Adresse locale	Usage
Application Squadro	http://localhost:8080	Accueil, lobby, plateau, actions de jeu.
Adminer	http://localhost:8081	Consultation de PostgreSQL en développement.
PostgreSQL	db:5432 depuis le réseau Docker	Stockage des joueurs et des parties persistantes.

Table 1 – Services exposés par l'environnement Docker.

2.3 Identifiants de développement

Connexion Adminer

```
Système : PostgreSQL
Serveur : db
Utilisateur : squadro_user
Mot de passe : password
Base : squadro_db
```

Ces identifiants sont adaptés à un environnement local de démonstration. Pour un déploiement public, ils devraient être remplacés par des secrets injectés hors dépôt.

2.4 Parcours fonctionnel

1. L'utilisateur saisit un nom sur l'écran d'accueil.

2. Le serveur crée ou récupère un joueur et initialise la session.
3. Le lobby propose une partie locale, une partie en attente ou le mode contre l'Oracle.
4. Le plateau affiche le tour courant, les pièces jouables, l'analyse tactique et l'historique.
5. Chaque action validée met à jour la session et, si une partie persistante existe, la base PostgreSQL.
6. La victoire clôture la partie et fige le vainqueur.

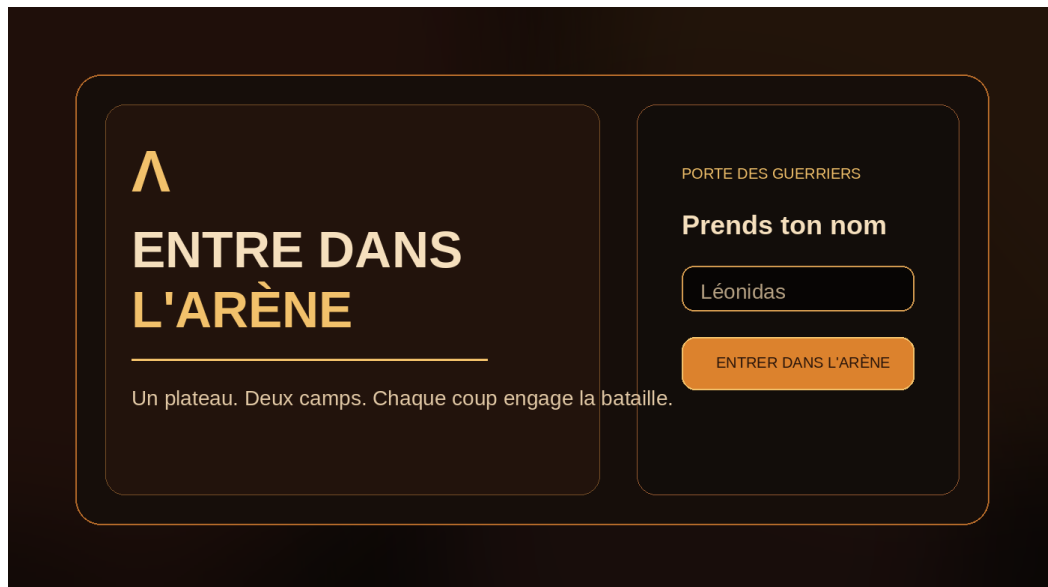


Figure 1 – Représentation de l'accueil spartiate : un point d'entrée simple, centre sur l'identité du joueur et l'entrée dans l'arène.

2.5 Raccourcis clavier

Raccourci	Effet
1 à 5	Joue l'une des pièces disponibles du camp actif.
O	Demande à l'Oracle de jouer un coup.
U	Annule le dernier coup si un état précédent est disponible.
F	Bascule le plateau en plein écran ou en mode focus.
M	Retourne au menu.
Entrée	Confirme le coup en mode confirmation.
Echap	Annule la confirmation courante.

Table 2 – Matrice des interactions clavier principales.

CHAPITRE 3

Regles du jeu et modelisation du domaine

3.1 Principe general

Squadro oppose deux camps : blanc et noir. Chaque camp dispose de cinq pieces. Une piece blanche suit une trajectoire horizontale, tandis qu'une piece noire suit une trajectoire verticale. La distance parcourue depend d'une vitesse attachee a la ligne ou a la colonne. Lorsqu'une piece arrive au bout de sa piste, elle change de direction ; lorsqu'elle revient a sa zone d'origine, elle est consideree comme sortie. La premiere couleur a sortir quatre pieces gagne.

Ce modele se prete bien a une representation objet minimale : la piece porte l'etat local, le plateau porte la collection des pieces et les constantes de vitesse, l'action applique le coup et l'analyseur calcule les consequences avant execution.

3.2 Etat d'une piece

Une piece doit fournir les informations necessaires au calcul d'un deplacement et a la sauvegarde de l'etat.

Information	Role	Consequence metier
Couleur	Blanc, noir ou neutre	Determine le sens de lecture du plateau et les interactions adverses.
Position	Coordonnees x, y	Identifie la case courante et l'origine d'un coup.
Direction	Aller ou retour	Selectionne la vitesse applicable et le sens du deplacement.
Statut jouable	Piece active ou sortie	Empêche de rejouer une piece terminee.
Serialization	Conversion JSON	Permet la persistance en session et en base.

Table 3 – Structure conceptuelle d'une piece Squadro.

3.3 Vitesses et trajectoires

Les vitesses sont distinctes entre l'aller et le retour. Cette asymetrie est au coeur du jeu : avancer vite peut exposer une piece, tandis qu'un retour plus court peut devenir decisif. Le moteur ne demande pas au joueur de saisir une distance ; il la deduit du plateau et de l'etat de la piece.

Invariant de deplacement

Pour une piece donnee, un coup legal doit satisfaire trois conditions : la piece appartient au camp actif, elle n'est pas deja sortie, et sa destination calculee reste compatible avec le bord, le retournement ou la sortie autorisee. Toute action utilisateur est revalidee cote serveur, meme si le bouton correspondant etait deja desactive cote interface.

3.4 Renvoi d'une piece adverse

Lorsqu'une piece traverse une case occupee par une piece adverse, l'adversaire est renvoye vers sa zone de depart logique. Cette regle introduit une dimension tactique importante. Elle est egalement utilisee par l'Oracle, qui valorise les coups capables de provoquer un recul adverse ou d'eviter une menace immediate.

CHAPITRE 4

Architecture logicielle

4.1 Lecture generale

Le projet adopte une architecture MVC pragmatique. Il ne cherche pas à reproduire la structure d'un framework complet, mais à séparer nettement les responsabilités : modèle métier, contrôleur d'action, vues et couche de persistance.

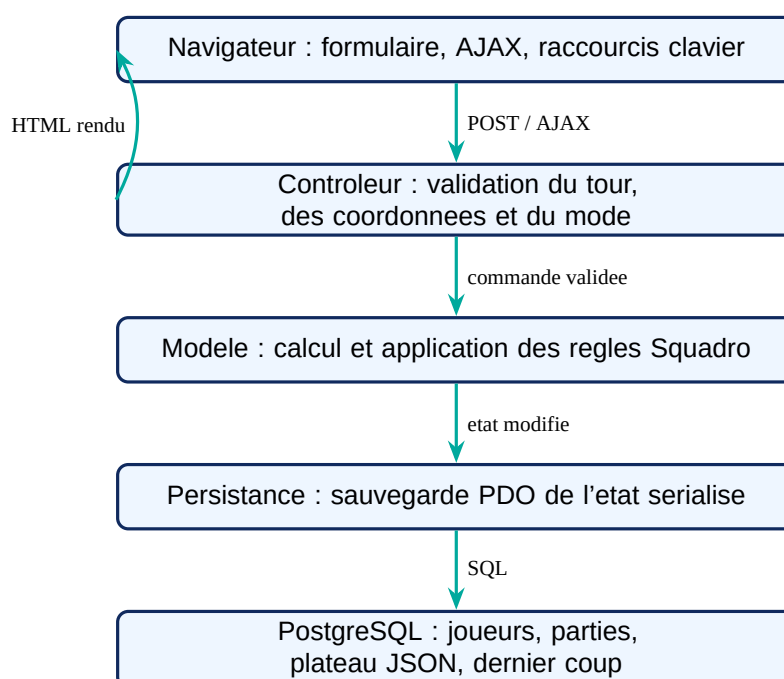


Figure 2 – Flux architectural principal : l'interface déclenche les actions, le contrôleur valide, le modèle applique, la persistance synchronise.

4.2 Arborescence commentée

Structure fonctionnelle du depot

```

squadro_game/
|-- Core/
|   |-- App.php           session, flash, settings, stats, securite
|   |-- bootstrap.php     chargement central des classes
|-- Contrôleur/
|   |-- index_squadro.php  entree du plateau
|   |-- traiteActionSquadro.php actions de jeu et orchestration
|-- Modele/
|   |-- piece_squadro.php  piece, couleur, direction, JSON
|   |-- plateau_squadro.php grille, vitesses, progression
|   |-- action_squadro.php application effective d'un coup
|   |-- SquadroAnalyzer.php analyse tactique et Oracle
|   |-- PieceSquadroUI.php rendu HTML du plateau et des panneaux
|   |-- joueurSquadro.php  entite joueur
|   |-- partieSquadro.php  entite partie persistante
|-- Vue/
|   |-- skel/PDOSquadro.skel.php acces PostgreSQL
|-- SQL/squadro.sql       schema rejouable
  
```

4. Architecture logicielle

```
|-- assets/css/           theme general, accueil, arene  
|-- assets/js/app.js      AJAX, raccourcis, stabilisation  
|-- tests/               lint et smoke tests  
|-- Dockerfile  
`-- docker-compose.yml
```

4.3 Role du bootstrap

Le fichier `Core/bootstrap.php` centralise les inclusions et l'initialisation minimale. Cette decision evite de multiplier les `require_once` dans chaque page, source classique d'erreurs de chemin. La classe `App` fournit ensuite les operations transversales : demarrage de session, messages flash, redirections, echappement HTML, parametres de jeu, statistiques et pile d'annulation.

Responsabilites principales de `App.php`

```
SESSION_PLAYER identite du joueur courant  
SESSION_BOARD plateau courant en session  
SESSION_TURN couleur active  
SESSION_HISTORY historique de bataille  
SESSION_UNDO pile d'annulation limitee  
SESSION_SETTINGS preferences de gameplay  
SESSION_STATS statistiques de la partie
```

4.4 Controleur principal

`Controlleur/traiteActionSquadro.php` est le point de passage des actions de jeu. Il ne dessine pas le plateau ; il verifie les entrees, appelle le moteur et synchronise l'etat. Cette concentration est acceptable car les fonctions internes restent specialisees : parsing d'un coup, verification de la couleur active, application d'un mouvement, export JSON, annulation, reglages et declenchement de l'Oracle.

CHAPITRE 5

Persistance PostgreSQL

5.1 Motivation

La persistance n'est pas seulement un confort. Elle permet de conserver une partie au-delà d'une requête HTTP, de partager une table entre deux joueurs et d'inspecter l'évolution du jeu. Le plateau est serialisé en JSON car il constitue un état compact, fortement structuré et rarement interrogé par morceaux.

5.2 Schema relationnel

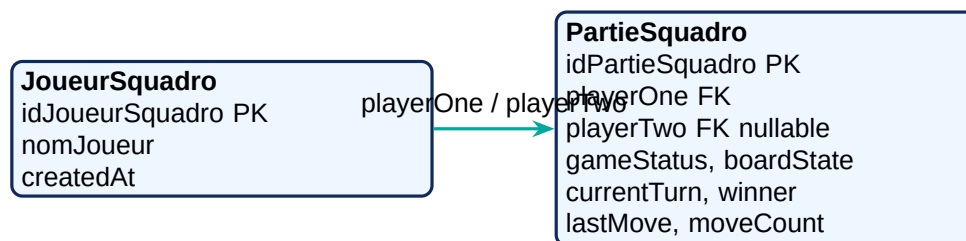


Figure 3 – Modèle relationnel minimal : une partie référence un ou deux joueurs et conserve l'état logique du plateau.

5.3 Etats d'une partie

Etat	Interpretation
initialized	Etat technique initial, avant exposition de la partie.
waitingForPlayer	Partie créée par un premier joueur et disponible pour un second.
active	Partie démarrée ou déjà jouable.
finished	Victoire détectée et vainqueur renseigné.

Table 4 – Cycle de vie persistant d'une partie Squadro.

5.4 Script SQL idempotent

Le fichier SQL/squadro.sql crée les tables et applique des ajouts de colonnes via `ADD COLUMN IF NOT EXISTS`. Cette approche est pratique dans un cadre Docker local : le script peut être relancé sans échouer sur des objets déjà présents. Les contraintes `CHECK` documentent les domaines autorisés pour le statut, le tour courant et le vainqueur.

Extrait conceptuel des contraintes SQL

```
gameStatus IN ('initialized', 'waitingForPlayer', 'active', 'finished')
currentTurn IN ('blanc', 'noir')
winner IS NULL OR winner IN ('blanc', 'noir')
```

5.5 Synchronisation après un coup

Après un coup valide, le contrôleur appelle une synchronisation. La base reçoit le statut, le plateau serialisé, le tour courant, le vainqueur éventuel, le dernier coup et le compteur de coups. Cette opération

5. Persistence PostgreSQL

reste volontairement simple : le serveur PHP est la source de verité pendant la requete, PostgreSQL devient le support de reprise.

Donnees synchronisees apres une action

- **Statut** : la partie reste active ou devient terminee si une victoire est detectee.
- **Plateau** : l'etat complet est converti en JSON afin d'etre reconstruit lors d'une reprise.
- **Tour courant** : la couleur active est conservee pour eviter une reprise ambiguë.
- **Dernier coup** : un libelle humain permet de comprendre la derniere transition sans relire tout le plateau.
- **Compteur** : le nombre de coups offre un repere simple pour l'historique et l'affichage.

Cette strategie evite de decomposer prematurement le plateau en de nombreuses lignes SQL. Le choix d'une colonne JSON reste acceptable tant que la base sert essentiellement de support de sauvegarde et de partage de partie. Si l'application devait produire des statistiques globales sur des milliers de parties, une normalisation supplementaire deviendrait pertinente : table des coups, table des evenements, index sur les sorties et calculs agregés par joueur.

CHAPITRE 6

Moteur de jeu et invariants

6.1 Application d'un coup

Un coup suit une sequence stricte : verification de la couleur active, verification de la piece, analyse previsionnelle, sauvegarde de l'etat precedent, application, mise a jour des statistiques, controle de victoire, puis changement de tour. Cette sequence limite les incoherences : le plateau n'est modifie qu'apres validation complete.

1. Reception du coup : origine, couleur, source manuelle ou Oracle.

2. Validation serveur : tour actif, mode en ligne, piece jouable, coordonnees.

3. Analyse tactique : destination, distance, recul, sortie, score heuristique.

4. Empilement de l'etat precedent pour l'annulation.

5. Application par ActionSquadro et mutation controlee du plateau.

6. Historique, statistiques, verification de victoire, synchronisation DB.

Figure 4 – Pipeline d'execution d'un coup.

6.2 Pile d'annulation

L'annulation conserve les derniers etats du plateau sous forme serialisee. Le nombre d'elements est limite a cinq. Ce choix evite une croissance non controlee de la session PHP et suffit pour corriger les erreurs de manipulation courantes.

6.3 Controle de victoire

La victoire est calculee a partir de l'etat du plateau. Lorsqu'une couleur a termine suffisamment de trajets, le controleur conserve la couleur gagnante et marque la partie comme terminee. Le dernier coup reste disponible pour l'affichage et l'audit de la partie.

6.4 Gestion des erreurs

Les erreurs utilisateur ne doivent pas produire un echec serveur. Une piece non jouable, une couleur incorrecte ou une position invalide genere un message flash et ramene l'application vers un etat coherent. Les exceptions non prevues sont capturees par le gestionnaire central pour produire une page d'erreur propre lorsque le mode debug est desactive.

CHAPITRE 7

Oracle tactique et enrichissement du gameplay

7.1 Raison d'être

La version de base d'un jeu de plateau peut rapidement sembler limitée si elle se contente d'appliquer les règles. L'Oracle ajoute une lecture tactique : il évalue les coups disponibles, en propose un prioritaire et peut jouer automatiquement pour un camp. Il donne également une valeur pédagogique au projet, car il sépare explicitement la règle de jeu de l'analyse stratégique.

7.2 Evaluation d'un coup

Chaque coup peut être décrit par des métadonnées : origine, destination, distance, reculs provoqués, sortie, changement de direction, résumé et score. Le score n'est pas une preuve de meilleur jeu ; c'est une heuristique lisible qui classe les options immédiates.

Facteur	Effet dans l'analyse
Distance parcourue	Valorise les coups qui font progresser une pièce.
Sortie immédiate	Tres fortement valorisée car elle rapproche directement de la victoire.
Recul adverse	Valorise l'impact défensif ou offensif du coup.
Demi-tour	Signale un changement de phase dans la trajectoire de la pièce.
Menace adverse	Aide à identifier une réponse défensive possible.

Table 5 – Elements pris en compte par l'analyse tactique.

7.3 Mode contre l'Oracle

En mode Oracle automatique, le joueur peut jouer les blancs pendant que l'Oracle répond pour les noirs. Cette fonctionnalité transforme le prototype en jeu solitaire exploitable, même en l'absence d'un second joueur. Le bot reste volontairement déterministic et local : il n'utilise pas de service externe, ne consomme pas d'API et ne dépend pas d'un modèle probabiliste.

7.4 Panneaux de jeu enrichis

L'interface expose les cinq meilleurs coups, l'historique et les statistiques. Cette présentation rend le jeu plus lisible : le joueur comprend ce qui vient de se produire, ce qui peut être tenté ensuite et quelle couleur domine la progression.

7. Oracle tactique et enrichissement du gameplay

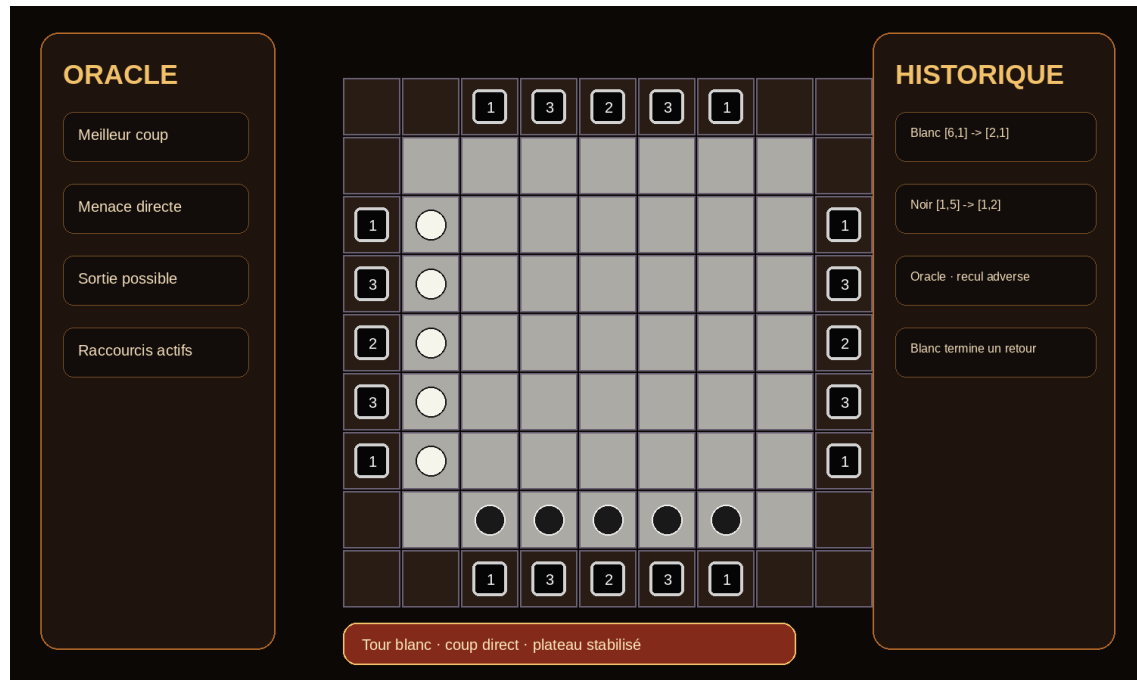


Figure 5 – Representation de l’écran de partie enrichi : plateau central, Oracle, historique, commandes rapides et stabilisation visuelle.

L’intérêt de ce panneau n’est pas seulement visuel. Il transforme l’état interne du moteur en informations immédiatement exploitables : le joueur voit son tour, les coups possibles, l’effet du dernier mouvement et l’avance relative des deux camps. Cette lisibilité réduit la charge cognitive et permet de concentrer l’attention sur la décision tactique.

Le choix d’une interface serveur enrichie par JavaScript natif reste cohérent avec le périmètre du projet : les règles continuent d’être appliquées par PHP, tandis que JavaScript se limite à l’interaction, aux raccourcis, à l’AJAX et à la stabilisation du viewport.

CHAPITRE 8

Interface, theme et stabilisation du plateau

8.1 Theme grec et spartiate

Le theme graphique emploie une palette sombre et metallique : noir obsidienne, bronze, rouge profond et ivoire. L'objectif n'est pas seulement decoratif ; il s'agit de donner une coherence aux ecrans. L'accueil devient une porte d'arene, le lobby une agora, le plateau une zone de bataille et la victoire une proclamation.

Les styles sont separees en trois fichiers :

- assets/css/app.css pour la base commune ;
- assets/css/sparta-home.css pour l'accueil ;
- assets/css/greek-game.css pour le lobby, le plateau et les panneaux de jeu.

Cette separation permet de changer l'identite visuelle sans modifier le moteur de regles.

8.2 Correction du defaut de scroll

Un defaut important venait du fait qu'apres un coup, le plateau pouvait changer de position dans la page. Ce comportement est particulierement nuisible dans un jeu de plateau : le joueur perd son repere spatial. La correction repose sur trois decisions complementaires.

1. Les actions courantes passent en AJAX afin d'eviter un rechargement complet.
2. Avant la requete, la position du plateau dans le viewport est memorisee.
3. Apres remplacement du shell de jeu, le script compense l'ecart vertical si le plateau a bouge.

Principe de stabilisation cote JavaScript

```
rememberBoardViewport();  
// envoi AJAX de l'action de jeu  
refreshGameShell(html);  
stabilizeBoardViewport();
```

8.3 Mode confirmation fixe

La confirmation n'est plus placee dans le flux bas de page. Elle est affichee dans une zone fixe, accessible sans defilement. En mode deplacement instantane, cette confirmation disparaît ; le clic applique directement le coup. Les deux modes coexistent pour respecter deux usages : rapidite ou validation explicite.

8.4 Accessibilite minimale

Le projet conserve une structure HTML standard : boutons, formulaires, labels, textes alternatifs lorsque c'est pertinent et raccourcis neutralises dans les champs de saisie. L'accessibilite n'est pas complete, mais la base evite les pieges les plus courants d'une interface entierement graphique.

CHAPITRE 9

Qualite, verification et maintenabilite

9.1 Strict types et centralisation

Les fichiers PHP principaux activent `declare(strict_types=1)`. Cette discipline ne transforme pas PHP en langage statiquement type, mais elle reduit les conversions implicites accidentelles. La centralisation du bootstrap limite egalement les erreurs de chargement, frequentes dans les projets PHP multi-repertoires.

9.2 Echappement des sorties

Toute valeur utilisateur destinee au HTML doit passer par `App::e`. Cette regle protege l'interface contre l'injection HTML simple. Les noms de joueurs, messages flash et elements variables du plateau sont donc echappes avant affichage.

9.3 Controle syntaxique

Verification PHP

```
find . -path './.git' -prune -o -name '*.php' -print0 | xargs -0 -n1 php -l
```

Verification JavaScript

```
node --check assets/js/app.js
```

9.4 Smoke tests

Le fichier `tests/smoke.php` joue le role de test de fume. Il verifie que les classes essentielles peuvent etre chargees et que le moteur de base reste exploitable. Le test ne remplace pas une suite complete, mais il permet de detecter rapidement une rupture de chargement ou une regression grossiere.

Execution des smoke tests

```
php tests/smoke.php
```

9.5 Limites de la couverture

La couverture de tests actuelle reste volontairement legere. Une prochaine version devrait introduire PHPUnit et separer les cas suivants : creation d'un plateau, deplacement blanc, deplacement noir, demi-tour, sortie, recul adverse, victoire, serialization JSON, reprise depuis PostgreSQL, undo et choix de l'Oracle.

CHAPITRE 10

Problemes corriges pendant l'evolution

10.1 Sortie avant les headers

Un probleme PHP classique provenait d'une sortie parasite avant `session_start()` ou `header()`. La correction a consiste a supprimer les balises fermantes finales des fichiers PHP purs et a retirer les lignes de test non proteges qui pouvaient produire du texte au chargement.

Erreur corrigee : headers already sent

La presence d'un caractere ou d'une ligne vide apres une balise `?>` suffit a envoyer une sortie HTTP. Les redirections et l'ouverture de session deviennent alors impossibles. La correction consiste a ne pas fermer les fichiers PHP purs et a centraliser le demarrage de session.

10.2 Serialization du plateau

La persistance necessite un format stable. Le plateau est converti en JSON et reconstruit lors de la reprise. Cette correction evite les etats partiellement sauvegardes ou les objets PHP non exploitables apres changement de requete.

10.3 Confirmation trop basse dans la page

La premiere interface demandait parfois de descendre pour confirmer un coup. Cette contrainte etait incompatible avec une experience de jeu fluide. Le mode direct et la barre fixe ont supprime ce frottement.

10.4 Plateau instable apres un coup

La stabilisation AJAX repond a un default plus subtil : meme sans bouton trop bas, un remplacement de page peut modifier la position visible du plateau. La correction conserve la position relative du plateau et evite au joueur de perdre son contexte visuel.

CHAPITRE 11

Bilan critique et perspectives

11.1 Bilan

Le projet aboutit à une version jouable et cohérente de Squadro. Les points les plus importants ne se situent pas uniquement dans l'ajout d'un thème graphique. La valeur principale du travail vient de la consolidation : environnement reproductible, contrôle des sessions, séparation des responsabilités, persistance, stabilisation de l'interface et enrichissement par une analyse tactique.

L'application reste volontairement simple sur certains aspects : pas de framework, pas de système de comptes complet, pas de WebSocket, pas de migration outillée. Ces absences ne sont pas nécessairement des défauts dans le cadre du projet ; elles rendent au contraire le code plus lisible et plus facilement évaluable.

11.2 Perspectives techniques

- remplacer l'attente par polling par des WebSockets ou Server-Sent Events ;
- introduire une authentification propre des joueurs ;
- ajouter un vrai matchmaking ;
- créer une page de replay coup par coup ;
- renforcer l'Oracle par une recherche minimax limitée ;
- remplacer le script SQL unique par un système de migrations versionnées ;
- ajouter une suite PHPUnit structurée ;
- prévoir un mode production avec secrets séparés, HTTPS et journalisation.

11.3 Apport pédagogique

Ce projet met en évidence une difficulté récurrente du développement web : un jeu n'est pas seulement une collection de pages. Il faut maintenir la cohérence d'un état, traiter correctement les actions concurrentes ou successives, conserver les données utiles et faire en sorte que l'interface ne gêne jamais la compréhension du plateau. C'est sur ce point que le travail est le plus formateur : la qualité perçue par le joueur dépend autant des détails d'ergonomie que de l'exactitude du moteur.

ANNEXE A

Annexe A - Matrice des fichiers principaux

Fichier	Couche	Responsabilite
Core/App.php	Transverse	Session, flash messages, settings, statistiques, undo, echappement HTML.
Core/bootstrap.php	Transverse	Chargement central des classes necessaires.
Vue/login.php	Vue	Accueil, saisie du joueur, entree dans le lobby.
Vue/choixAction.php	Vue	Lobby principal et choix du mode de jeu.
Contrôleur/traitementActionSquadro.php	Contrôleur	Reception, validation et execution des actions de jeu.
Modele/piece_squadro.php	Modele	Représentation d’une piece et conversion JSON.
Modele/plateau_squadro.php	Modele	Grille, vitesses, progression et serialization du plateau.
Modele/action_squadro.php	Modele	Application concrete des coups et verification de victoire.
Modele/SquadroAnalyzer.php	Service metier	Analyse des coups legaux, score tactique et Oracle.
Modele/PieceSquadroUI.php	Vue metier	Rendu HTML du plateau, panels, dock et commandes rapides.
skel/PDOSquadro.skel.php	Persistence	Requetes SQL, creation de joueurs, parties, sauvegarde et reprise.
SQL/squadro.sql	Donnees	Schema PostgreSQL, contraintes et index.
assets/js/app.js	Frontend	AJAX, raccourcis, plein ecran, previews, stabilisation du plateau.
assets/css/greek-game.css	Frontend	Theme global grec/spartiate.

Table 6 – Role des principaux fichiers du projet.

ANNEXE B

Annexe B - Commandes de verification

Cycle de verification recommande

```
# Verification syntaxique PHP
find . -path './.git' -prune -o -name '*.php' -print0 | xargs -0 -n1 php -l

# Verification syntaxique JavaScript
node --check assets/js/app.js

# Smoke tests du moteur
php tests/smoke.php

# Demarrage Docker
docker compose down -v
docker compose up --build
```

ANNEXE C

Annexe C - Source institutionnelle du logo

Le logo institutionnel utilise dans ce rapport remplace la marque Sorbonne du template de reference par une marque Université Le Havre Normandie. La page officielle de l'établissement précise que le logo-type est l'élément central de l'identité visuelle et propose des téléchargements adaptés à l'affichage écran et à l'impression. Elle rappelle également que les supports largement diffusés doivent être validés par la Direction de la Communication.

Reference

Page officielle : <https://www.univ-lehavre.fr/fr/universite/informations-pratiques/identite-visuelle-et-logo/>